Strategic Plan: Cloud Migration to AWS

ACME Cloud Corp — Legacy Java Application Migration to AWS

Prepared by: Ernest Foubi (ernfou7189)

Project Manager, ECPI Consulting and Programming Institution

CYB350 — Introduction to DevOps and Project Management

Instructor: Samuel Liberatore

November 2026

Contents

Executive Summary **3**

Part 1: Scrum Team Agreement **4**

1.1 Purpose and Scope 4
1.2 Definition of Ready 4
1.3 Definition of Done 5
1.4 Meeting Cadence 6
1.5 Performance Measures 7
1.6 Additional Topics From the Weekly Discussion 8
1.7 Maintaining This Agreement 10

Part 2: Asset Identification and CI/CD Plan **11**

2.1 Purpose and Scope 11
2.2 Asset Inventory, Cost, and Sourcing Responsibility 11
2.3 Required Resources and Instance Counts 13
2.4 CI/CD Pipeline Outline 14
2.5 Cost Controls and Ownership 15
2.6 Risks and Next Decisions 15

Part 3: Impact Outline **16**

3.1 Summary of the Business Case 16
3.2 What the Cloud Migration Project Is 16
3.3 Project Phases and Expected Deliverables 17
3.4 Five Benefits for ACME 18
3.5 How ACME Will Measure Success 20
3.6 Recommendation 21

Part 4: Security Assessment **22**

4.1 Project Overview 22
4.2 Technologies and Architecture in Scope 23
4.3 Threat Model 24
4.4 Security Policies 27
4.5 Incident Response Plan 29

References **33**

Executive Summary

ACME Cloud Corp runs its legacy Java web application on company-owned servers and builds and deploys every release by hand. This strategic plan, prepared by ECPI Consulting and Programming Institution, moves the application and its database to Amazon Web Services (AWS) and replaces manual releases with an automated, secure CI/CD pipeline built on AWS CodePipeline. As the client requires, the application runs on Amazon EC2, the database on Amazon RDS, and access is controlled through AWS IAM.

The plan has four parts. **Part 1, the Scrum Team Agreement,** sets how the team works: its Definition of Ready and Definition of Done, meeting cadence, and performance measures. **Part 2, Asset Identification and CI/CD Plan,** lists the 22 assets the project needs, their costs and owners, and the 12-stage pipeline that moves code from commit to production. **Part 3, the Impact Outline,** explains the project phases, deliverables, and five benefits ACME can expect. **Part 4, the Security Assessment,** identifies five threats, the mitigations and security policies that address them, and the incident response plan the team will follow.

The project is delivered in **12 weeks** over six two-week sprints. The full environment, including the security services recommended in Part 4, costs about **$5,604 per year**, roughly 22% of ACME's $25,000 annual budget, leaving more than $19,000 as contingency for growth and high-availability upgrades. We recommend that ACME approve the plan and begin discovery and planning in the next sprint.

Part 1: Scrum Team Agreement

1.1 Purpose and Scope

This Scrum Team Agreement sets the working standards for the Scrum team migrating ACME Cloud Corp's legacy Java web application from a manually built, on-premises deployment to Amazon Web Services (AWS). The target environment hosts the application on Amazon EC2, stores data in Amazon RDS, manages access through AWS IAM, and automates build and deployment through AWS CodePipeline, all within a $25,000 annual budget.

The agreement gives the team a shared definition of what "ready" and "done" mean, how often and how the team meets, how progress and quality will be measured, and how team members communicate and hold one another accountable. As Schwaber and Sutherland (2020) explain, Scrum depends on transparency, inspection, and adaptation; this document makes those expectations explicit so the team can inspect and improve them every sprint.

Team Composition and Sprint Length

Role	Count	Primary focus
Product Owner (ACME representative)	1	Owns and prioritizes the backlog; accepts completed work
Scrum Master / Project Manager	1	Facilitates events, removes blockers, protects the agreement
Java Developers	3	Refactor and containerize-ready the application for EC2; RDS data access
DevOps / Cloud Engineer	1	CodePipeline, IAM roles and policies, EC2/RDS provisioning
QA / Test Engineer	1	Automated and exploratory testing, defect tracking

The team will work in **two-week sprints** and track all work items in Jira.

1.2 Definition of Ready

A product backlog item may be pulled into a sprint only when it meets all of the following criteria. This prevents the team from starting work that is unclear, too large, or blocked.

- The user story follows the format “As a [user], I want [goal], so that [benefit]” and has a clear business value.
- Acceptance criteria are written, testable, and agreed upon by the Product Owner.
- The item is estimated in story points and is small enough to finish within one sprint (8 points or fewer).
- Dependencies (e.g., IAM permissions, RDS schema changes, third-party access) are identified and either resolved or scheduled.
- Any security, compliance, or cost impact on the AWS environment has been noted on the ticket.

1.3 Definition of Done

A product backlog item is considered **done** only when every criterion below is met. Each criterion is measurable and is verified by a specific person or tool, so “done” never depends on opinion. If any criterion is not met, the item returns to the backlog and is not counted toward the sprint’s completed work.

#	Done criterion	How it is verified
1	**Peer reviewed:** At least two team members have reviewed and approved the pull request.	Two approvals recorded in the repository before merge to main
2	**Tested:** All unit and integration tests pass, and new code maintains at least 80% unit-test coverage.	Test and coverage report generated in the CodePipeline build stage
3	**Builds in the pipeline:** The change merges to main and completes every AWS CodePipeline stage (source, build, test, deploy to staging) without manual steps.	Green pipeline execution in the CodePipeline console
4	**Security checked:** Static code and dependency scans report zero critical or high-severity findings; no credentials are hard-coded (secrets are stored in AWS Secrets Manager).	Scan report attached to the ticket; reviewer confirms no secrets in code
5	**Least-privilege access:** Any new or changed IAM role or policy grants only the permissions the feature needs and has been reviewed by the DevOps engineer.	IAM policy reviewed and noted on the ticket
6	**Data changes are safe:** Any RDS schema or data migration script has been run successfully in staging and has a tested rollback script.	Migration and rollback both executed in staging
7	**Accepted:** The Product Owner confirms all acceptance criteria are met.	PO sign-off recorded in Jira
8	**Documented and demo-ready:** The README, runbook, or architecture notes are updated, and the feature can be demonstrated at the Sprint Review.	Documentation link on the ticket; feature shown in staging
9	**Cost-aware:** New AWS resources are tagged with the project and environment so spend can be tracked against the $25,000 annual budget.	Tags visible in AWS Cost Explorer / AWS Budgets

The Definition of Done is owned by the whole team. It may be tightened during a Sprint Retrospective (for example, raising the coverage target), but it may not be relaxed mid-sprint to make an item appear complete.

1.4 Meeting Cadence

Daily Standup

Element	Agreement
Frequency	Every working day, Monday through Friday
Time	9:30 a.m. Eastern, so the team starts the day aligned and blockers surface early
Duration	15 minutes maximum (strict timebox)
Format	Hybrid: in person or on Microsoft Teams with cameras on, standing in front of the Jira sprint board
Facilitator	Scrum Master; the team rotates who shares the board to keep everyone engaged
Agenda	Each member answers: What did I complete toward the sprint goal? What will I work on today? What is blocking me?
Ground rules	Start on time; no problem-solving during standup; detailed discussions go to a “parking lot” right after the meeting with only the people who need to be there
Absences	A member who cannot attend posts a written update in the team Teams channel before 9:30 a.m.

Other Scrum Events (per two-week sprint)

Timeboxes below are scaled from the maximums in the Scrum Guide for a one-month sprint (Schwaber & Sutherland, 2020).

Event	When	Timebox	Purpose
Sprint Planning	Day 1 of the sprint	3 hours	Agree on the sprint goal and select Ready backlog items
Backlog Refinement	Weekly, Wednesday	1 hour	Break down and estimate upcoming stories so they meet the Definition of Ready
Sprint Review	Last day of the sprint	1.5 hours	Demonstrate Done work to ACME stakeholders and gather feedback
Sprint Retrospective	Last day, after the Review	1.5 hours	Review performance measures and this agreement; choose one or two improvements

This cadence is realistic for a seven-person team: it totals roughly 2.5 hours of standups and about 8 hours of other events per two-week sprint, leaving most time for delivery while still giving stakeholders a predictable rhythm of feedback.

1.5 Performance Measures

The team will track four measures that together show whether the migration is progressing predictably and whether the software being delivered is high quality. Data comes from tools the team already uses (Jira and AWS CodePipeline), so measurement adds little overhead. Results are reviewed at every Sprint Retrospective and are used to improve the team's process, not to rank or criticize individuals.

Measure	How it is calculated	Data source	Target
1. Sprint commitment reliability	Story points completed (meeting the DoD) ÷ story points committed at Sprint Planning × 100	Jira sprint report	85% or higher each sprint
2. Pipeline success rate	Successful CodePipeline executions ÷ total executions × 100	AWS CodePipeline execution history	90% or higher; any failed build fixed within 1 business day
3. Escaped defect rate	Number of defects found in production (or by ACME after release) per sprint	Jira bugs tagged "escaped"	2 or fewer per sprint, trending downward
4. Blocker resolution time	Average hours from when a blocker is raised at standup to when it is cleared	Jira "Blocked" status timestamps	24 hours or less on average

Why These Measures

- **Commitment reliability** shows whether the team plans realistically, which keeps the migration timeline credible for ACME.
- **Pipeline success rate** and **escaped defect rate** measure delivery quality. They mirror widely used DevOps delivery metrics such as change failure rate (Forsgren et al., 2018) and confirm that automating deployment through CodePipeline is reducing risk compared with the old manual process.
- **Blocker resolution time** measures how well the team communicates and supports one another, connecting directly to the communication agreement in Section 1.6.

If a measure misses its target for two sprints in a row, the team will choose a specific improvement action at the retrospective and track it as a backlog item.

1.6 Additional Topics From the Weekly Discussion

In the Week 2 discussion on Agile team agreements, two themes stood out as essential for a software team: clear communication and collaboration, and well-defined roles with shared accountability. Both are added to this agreement because a cloud migration depends on developers, operations, testers, and the client staying tightly coordinated.

Topic 1: Communication and Collaboration

Effective communication keeps developers, testers, operations staff, and stakeholders informed about progress, requirements, and challenges, and it prevents misunderstandings, duplicate work, and delays. The team agrees to the following norms:

- **Raise blockers immediately.** Any blocker is raised at the next standup or, if urgent, posted in the team channel right away and tagged to the Scrum Master. Blockers not cleared within one business day are escalated to the Product Owner or ACME stakeholders.
- **Use the right channel.** Microsoft Teams chat for quick questions; Jira comments for anything about a specific work item, so decisions stay with the ticket; email only for formal client communication.
- **Response expectations.** Team messages are acknowledged within 4 working hours; pull-request review requests are picked up within 1 business day.
- **Constructive feedback.** Code-review comments focus on the code, not the person, and explain the reason behind a requested change.
- **Transparency for the client.** The Jira board and CodePipeline status are visible to ACME, and Sprint Reviews are open to its stakeholders.

Compliance is measured through the blocker resolution time measure in Section 1.5, the number of unresolved blockers at the end of each sprint, and review turnaround time on pull requests.

Topic 2: Roles, Responsibilities, and Accountability

Every team member should understand their responsibilities and commitments throughout the sprint. Clear ownership prevents confusion about who is doing what, which is especially important when application, database, and infrastructure work overlap. The team agrees to the following:

- **Every work item has one owner.** Each Jira ticket is assigned to a single person who is accountable for moving it to Done, even when others help.
- **Acceptance criteria are defined up front,** as required by the Definition of Ready, so everyone knows what "finished" means before work begins.
- **Shared ownership of quality.** Developers are responsible for tests on their own code; QA owns the overall test strategy; the DevOps engineer owns the pipeline and IAM standards; everyone is responsible for meeting the Definition of Done.
- **Commitments are made by the team.** The team, not the project manager, decides how much work to commit to at Sprint Planning, which builds ownership of the sprint goal.

Activity	Product Owner	Scrum Master	Developers	DevOps Eng.	QA
Prioritize backlog	Accountable	Supports	Consulted	Consulted	Consulted
Write and meet acceptance criteria	Accountable	—	Responsible	Responsible	Responsible
Maintain CodePipeline and IAM	Informed	Supports	Consulted	Accountable	Consulted
Test strategy and defect tracking	Informed	—	Responsible	Consulted	Accountable
Remove blockers / facilitate events	Consulted	Accountable	Informed	Informed	Informed

Accountability is measured through sprint commitment reliability (Section 1.5), the number of completed items that pass acceptance criteria on the first review, and the number of tasks delayed because of unclear ownership, which the team reviews at each retrospective.

1.7 Maintaining This Agreement

This agreement is a living document. The team will review it at each Sprint Retrospective, and any member may propose a change. Changes are adopted by team consensus and recorded with the date. New team members review and sign the agreement during onboarding.

Name	Role	Signature	Date
	Product Owner (ACME)		
Ernest Foubi	Scrum Master / Project Manager		
	Developer		
	Developer		
	Developer		
	DevOps / Cloud Engineer		
	QA / Test Engineer		

Part 2: Asset Identification and CI/CD Plan

2.1 Purpose and Scope

This part identifies the assets required to migrate ACME Cloud Corp's legacy Java web application from a manually built, on-premises deployment to Amazon Web Services (AWS). It provides each asset's expected cost and sourcing responsibility, and outlines the CI/CD pipeline that moves code from a developer's commit to the cloud-hosted production environment.

It builds on the Scrum Team Agreement in Part 1: the same Scrum team owns the work, the pipeline enforces the same Definition of Done, and every resource is tagged to track spending against ACME's **$25,000 annual budget**. As required by the client, the solution hosts the application on Amazon EC2, stores data in Amazon RDS, manages authentication and authorization through AWS IAM, and automates deployment with AWS CodePipeline.

Planning Assumptions

Item	Assumption
AWS Region	US East (N. Virginia)
Operating hours	730 hours per month (always on) for production and staging
Workload	Moderate traffic and storage for an internal-to-midsize business web application
Pricing basis	AWS published on-demand pricing, rounded up to whole dollars
Budget scope	The $25,000 cap covers annual cloud operating costs; existing staff salaries are funded separately by ECPI and ACME

2.2 Asset Inventory, Cost, and Sourcing Responsibility

The project requires **22 assets** across cloud infrastructure, networking, security, pipeline tooling, monitoring, people, and policy. Each asset includes its expected monthly and annual cost, along with the role responsible for sourcing it. Sourcing roles are drawn from the Scrum team in Part 1, with shared ECPI specialists (Security Engineer, Database Administrator, and Finance) engaged as needed rather than added as permanent headcount.

#	Asset / purpose	Category	Monthly	Annual	Sourced by
1	Production EC2 instance (Linux) — hosts the Java application	Cloud infrastructure	$65	$780	DevOps / Cloud Engineer
2	Staging EC2 instance (Linux) — pre-production testing	Cloud infrastructure	$35	$420	DevOps / Cloud Engineer
3	Amazon RDS database instance (single-AZ)	Cloud infrastructure	$95	$1,140	DevOps / Cloud Engineer with DBA
4	RDS storage and automated backups	Cloud infrastructure	$35	$420	DevOps / Cloud Engineer with DBA
5	Application Load Balancer — HTTPS entry point and health checks	Cloud infrastructure	$22	$264	DevOps / Cloud Engineer
6	Amazon VPC, subnets, route tables, and security groups	Network	$0	$0	DevOps / Cloud Engineer
7	Public IPv4 addresses and data transfer	Network	$50	$600	DevOps / Cloud Engineer
8	AWS IAM roles, users, and policies	Security	$0	$0	Security Engineer / DevOps Engineer
9	AWS Secrets Manager — database and pipeline secrets	Security	$10	$120	Security Engineer / DevOps Engineer
10	TLS certificate via AWS Certificate Manager (attached to the load balancer)	Security	$0	$0	Security Engineer / DevOps Engineer
11	GitHub repository (source control)	Pipeline and tooling	$0	$0	Java Lead / Project Manager
12	AWS CodePipeline — pipeline orchestration	Pipeline and tooling	$15	$180	DevOps / Cloud Engineer
13	AWS CodeBuild — compile, test, and scan	Pipeline and tooling	$30	$360	DevOps / Cloud Engineer
14	AWS CodeDeploy — deployment to EC2	Pipeline and tooling	$0	$0	DevOps / Cloud Engineer
15	Amazon S3 — build artifact and backup storage	Pipeline and tooling	$15	$180	DevOps / Cloud Engineer
16	Amazon CloudWatch — logs, metrics, and alarms	Monitoring	$30	$360	DevOps / Cloud Engineer and QA
17	AWS Budgets — cost alerts and governance	Monitoring	$5	$60	Project Manager / Finance
18	Jira Software — backlog and sprint tracking (free tier, up to 10 users)	Pipeline and tooling	$0	$0	Project Manager / Scrum Master
19	Scrum team: Product Owner, Scrum Master, 3 Java developers, DevOps Engineer, QA	People	Existing salary	Outside cloud budget	ECPI and ACME Management

#	Asset / purpose	Category	Monthly	Annual	Sourced by
20	Shared specialists: Security Engineer, DBA, Finance (as needed)	People	Existing salary	Outside cloud budget	ECPI Management
21	Week 2 Team Agreement and Definition of Done	Policy	$0 (team time)	$0 (team time)	Project Manager / Scrum Master
22	Incident response plan and deployment runbook	Policy	$0 (team time)	$0 (team time)	Security Engineer / DevOps Engineer
	Total recurring cloud cost		**$407**	**$4,884**	

Budget Summary

Measure	Amount
Estimated monthly cloud cost	$407
Estimated annual cloud cost	$4,884
ACME annual budget cap	$25,000
Remaining headroom (contingency)	**$20,116**

The baseline uses only about 20% of the budget, leaving room for traffic growth, high-availability upgrades like Multi-AZ RDS or a second production instance, AWS Business Support if ACME needs it, and the security services recommended in Part 4, all while staying under the $25,000 cap.

2.3 Required Resources and Instance Counts

The client requires EC2, IAM, and RDS, and leaves the number of instances to the project manager. The table below states each count and the reason for it.

Resource	Instances	Rationale
Amazon EC2	2 (1 production, 1 staging)	Production hosts the Java application behind the load balancer. Staging mirrors production so every release is tested before it reaches users. A single production instance keeps cost low; the load balancer lets more instances be added later without redesign.
Amazon RDS	1 (single-AZ)	Stores application data in a private subnet, reachable only from the application's security group. Automated backups support recovery; Multi-AZ can be enabled later within budget.
AWS IAM	4 service roles + named user accounts	Separate least-privilege roles for CodePipeline, CodeBuild, CodeDeploy, and the EC2 instance. Each team member signs in with an individual account with multi-factor authentication; no shared or root credentials are used.
Application Load Balancer	1	Terminates HTTPS with a free ACM certificate and routes traffic only to healthy EC2 instances.

Resource	Instances	Rationale
AWS CodePipeline	1 pipeline	Orchestrates source, build, staging deployment, approval, and production deployment.

2.4 CI/CD Pipeline Outline

The pipeline replaces ACME's manual build and deployment process. Code progresses through the following stages in order, and a failure at any gate halts the release before it reaches production. Each automated gate aligns with a criterion in the Definition of Done (Section 1.3).

GitHub → CodePipeline → CodeBuild → S3 artifact → CodeDeploy (staging) → Approval → CodeDeploy (production) → CloudWatch

#	Stage	What happens	Gate to move forward
1	**Plan**	The Product Owner and developers refine a Jira story until it meets the Definition of Ready.	Story meets Definition of Ready
2	**Code and review**	Developers work on GitHub feature branches and open a pull request into the protected main branch.	Two peer approvals
3	**Source (CodePipeline)**	Merging to main triggers AWS CodePipeline through its GitHub connection, which fetches the code and starts a versioned run.	Merge to main
4	**Build (CodeBuild)**	CodeBuild compiles and packages the Java application with Maven, producing a versioned JAR/WAR deployment bundle.	Build succeeds
5	**Test and scan (CodeBuild)**	Unit and integration tests, coverage check, static code and dependency scans, and secret detection run automatically.	All tests pass; 80%+ coverage on new code; no critical/high findings
6	**Store artifact (S3)**	The tested bundle is saved to an encrypted S3 bucket. The same artifact is used for staging and production.	Artifact stored and versioned
7	**Deploy to staging (CodeDeploy)**	CodeDeploy installs the artifact on the staging EC2 instance, which uses test data and the staging database.	Deployment succeeds
8	**Validate staging**	QA runs smoke, functional, and security tests and tests any database migration and its rollback. Evidence is attached to Jira.	QA sign-off
9	**Approve release**	The Product Owner confirms the acceptance criteria and approves the manual approval action in CodePipeline.	Product Owner approval
10	**Deploy to production (CodeDeploy)**	CodeDeploy deploys the same tested artifact to the production EC2 instance behind the load balancer. Database changes run from reviewed scripts after a backup.	Load balancer health checks pass
11	**Monitor and recover**	CloudWatch tracks errors, instance health, and logs. If alarms fire, CodeDeploy rolls back to the last known-good version.	No active alarms
12	**Close and improve**	The runbook is updated, the work is shown at the Sprint Review, and pipeline metrics are reviewed at the Retrospective.	Item meets Definition of Done

The only manual step is the Product Owner's approval in stage 9. Everything else, from build through deployment and rollback, is automated, which removes the error-prone manual deployments ACME uses today.

2.5 Cost Controls and Ownership

- **Project Manager / Finance:** set AWS Budgets alerts at 50%, 80%, and 100% of the monthly forecast, review actual spend monthly, and approve any new resource before it is created.
- **DevOps / Cloud Engineer:** tag every resource by project and environment, right-size EC2 and RDS based on CloudWatch utilization, apply S3 and log retention rules, and remove unused resources.
- **Security Engineer:** review IAM least privilege, secrets handling, encryption, and scan results before each production release.
- **Developers and QA:** keep the pipeline green and ensure every item meets the Definition of Done before the Product Owner accepts it.

2.6 Risks and Next Decisions

Risk or open decision	Plan
Single production instance is a point of failure	Monitor uptime; add a second instance behind the load balancer if availability targets require it (fits within headroom).
Database engine and size not yet confirmed	Confirm engine and data volume with ACME during discovery; adjust the RDS instance class if needed.
Actual traffic may differ from assumptions	Validate estimates in the AWS Pricing Calculator and review AWS Budgets after the first month.
Security risks across the pipeline and environment	Addressed in detail in the Security Assessment (Part 4).

Part 3: Impact Outline

3.1 Summary of the Business Case

ACME Cloud Corp runs its core Java web application on servers it owns, and every new version is built and deployed by hand. This approach is slow, depends on a few people, and puts the business at risk each time a change goes live. This part makes the case for moving the application to Amazon Web Services (AWS) using the plan in this document: a Scrum team working under a clear team agreement, a defined set of cloud assets, and an automated CI/CD pipeline built on AWS CodePipeline.

The investment is small and predictable. The planned AWS environment costs about **$4,884 per year**, roughly 20% of ACME's $25,000 annual budget, leaving **$20,116** as contingency. The project takes **12 weeks** in six two-week sprints. In return, ACME can expect five benefits: lower and more predictable costs, faster and safer software releases, higher availability with faster recovery, stronger security, and room to scale as the business grows. Each benefit is tied to a measurable target so leadership can see whether the investment is paying off.

3.2 What the Cloud Migration Project Is

The project moves ACME's legacy Java web application and its database from the on-premises data center to AWS, and replaces manual builds and deployments with an automated pipeline. The client's requirements shape the design: Amazon EC2 hosts the application, Amazon RDS runs the database, AWS Identity and Access Management (IAM) controls who and what can access each resource, and AWS CodePipeline automates the path from code to production. Table 1 compares how ACME operates today with how it will operate after the migration.

Table 1. Current state compared with the future state

Area	Today (on-premises)	After migration (AWS)
Hosting	Company-owned servers that must be bought, powered, and replaced	EC2 instances paid for monthly, resized or replaced in minutes
Database	Self-managed database server and manual backups	Amazon RDS with automated backups and patching

Area	Today (on-premises)	After migration (AWS)
Deployments	Built and copied to servers by hand	Automated build, test, and deployment through CodePipeline
Testing	Inconsistent, often done after release	Automated tests and security scans on every change, plus a staging environment
Access control	Shared accounts and local credentials	Individual IAM accounts with MFA and least-privilege roles; secrets in Secrets Manager
Recovery	Manual troubleshooting and restore	Health checks, alarms, and one-step rollback to the last good version

3.3 Project Phases and Expected Deliverables

The work is organized into six phases delivered over six two-week sprints, following the cadence in the Scrum Team Agreement (Part 1). Each phase ends with a concrete deliverable that the Product Owner reviews at the Sprint Review, so ACME sees progress every two weeks rather than waiting until the end. Figure 1 shows the timeline, and Table 2 lists what ACME receives from each phase.

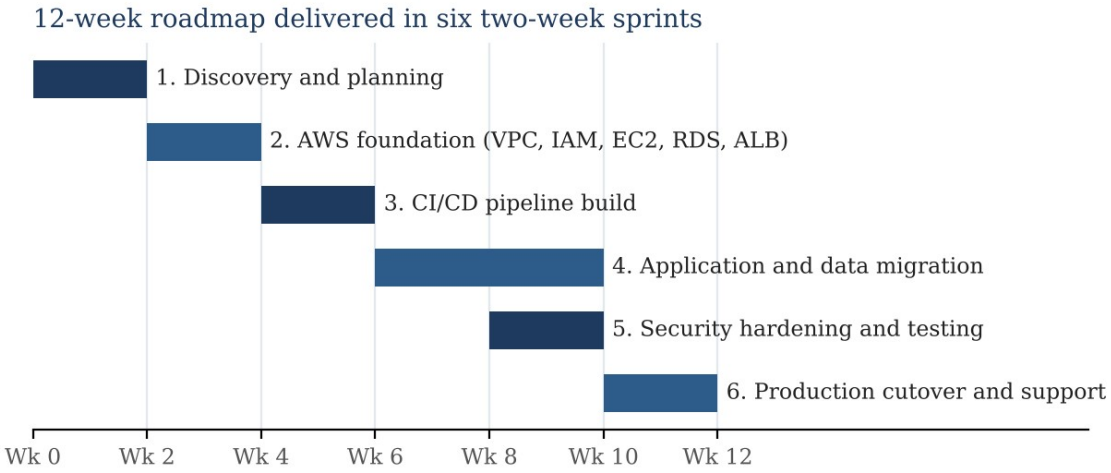


Figure 1. Twelve-week implementation roadmap

Table 2. Phases, timing, and deliverables

Phase	Weeks	Deliverables to ACME
1. Discovery and planning	1–2	Confirmed requirements, database engine and size, Scrum Team Agreement, prioritized product backlog
2. AWS foundation	3–4	Secure network (VPC and security groups), IAM roles and user accounts, EC2 staging and production instances, RDS database, load balancer with HTTPS
3. CI/CD pipeline build	5–6	Working CodePipeline with CodeBuild, automated tests, security scans, S3 artifact storage, and CodeDeploy to staging
4. Application and data migration	7–10	Java application running in staging, database migrated and validated, rollback scripts tested

Phase	Weeks	Deliverables to ACME
5. Security hardening and testing	9–10	Security assessment and fixes, encryption and secrets verified, load and acceptance testing complete
6. Production cutover and support	11–12	Application live on AWS, CloudWatch monitoring and alarms, runbook and incident response plan, team handover and two weeks of close support

Alongside the working system, ACME receives this complete strategic plan: the Scrum Team Agreement, the asset inventory and CI/CD plan, this impact outline, and the security assessment. Together these documents let ACME run, maintain, and improve the new environment after the project ends.

3.4 Five Benefits for ACME

Independent research shows what organizations typically gain from moving to AWS and from adopting automated delivery practices. An IDC study of companies that moved workloads to AWS found large average improvements in cost, downtime, and speed (Carvalho & Marden, 2018), as summarized in Figure 2. The five benefits below apply these findings to ACME's plan.

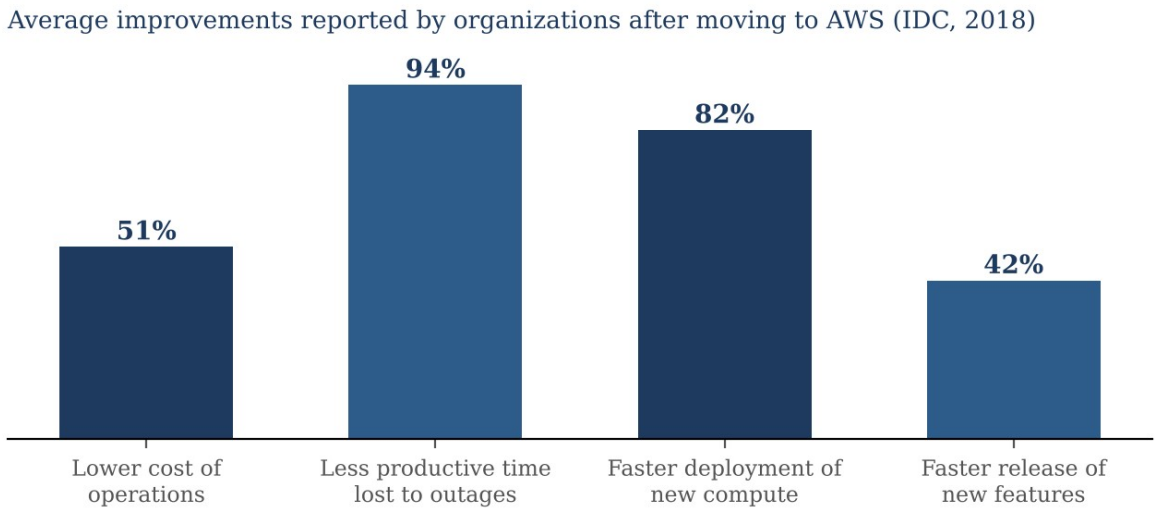


Figure 2. Average improvements reported after moving to AWS. Data from Carvalho and Marden (2018).

Benefit 1: Lower and More Predictable Costs

ACME stops buying, powering, and replacing its own servers and pays only for what it uses. The planned environment costs about $407 per month, and AWS Budgets alerts warn the team before spending drifts. IDC found that organizations running on AWS lowered their cost of operations by an average of 51%

compared with on-premises environments (Carvalho & Marden, 2018). Figure 3 shows how comfortably the plan fits within ACME's budget.

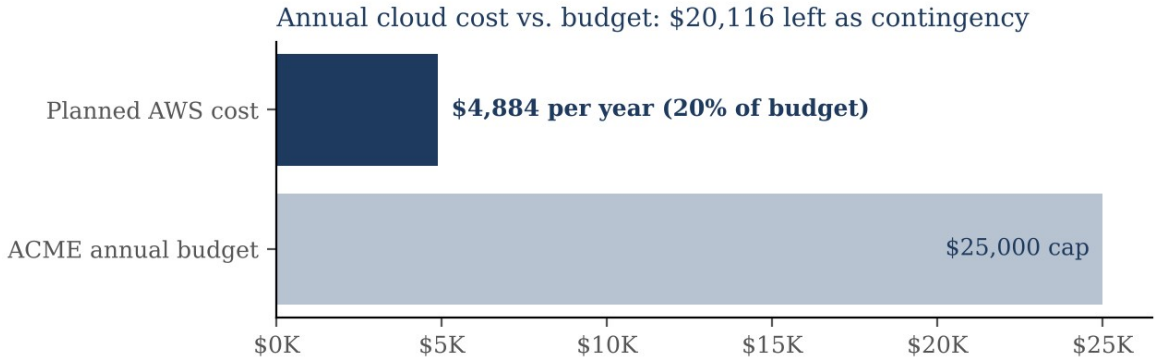


Figure 3. Planned annual AWS cost compared with ACME's $25,000 budget

Benefit 2: Faster and Safer Software Releases

Today, every release depends on manual steps that are slow and easy to get wrong. With CodePipeline, every approved change is built, tested, scanned, and deployed the same way every time. Research by DevOps Research and Assessment (DORA) found that elite teams using these practices deploy 208 times more often than low performers, move code from commit to production 106 times faster, and have a change failure rate seven times lower (Forsgren et al., 2019). IDC also found that AWS customers released new features 42% faster (Carvalho & Marden, 2018). For ACME, this means new features and fixes reach customers in days instead of weeks.

Benefit 3: Higher Availability and Faster Recovery

The load balancer checks the application's health and sends traffic only to working servers, CloudWatch alarms alert the team as soon as something fails, and CodeDeploy can roll back to the last good version in one step. RDS takes automated backups so data can be restored to a recent point in time. IDC reported 94% less productive time lost to unplanned outages after moving to AWS (Carvalho & Marden, 2018), and DORA found that elite teams recover from incidents 2,604 times faster than low performers (Forsgren et al., 2019). Less downtime protects ACME's revenue and its reputation with customers.

Benefit 4: Stronger Security

Under the AWS shared responsibility model, AWS secures the physical data centers, hardware, and core infrastructure, while ACME controls who can access its application and data (Amazon Web Services, n.d.-i). The plan uses individual IAM accounts with multi-factor authentication, least-privilege roles for every service, secrets stored in AWS Secrets Manager instead of in code, encrypted storage, and HTTPS through AWS Certificate Manager. Every change is scanned for vulnerabilities before release. The Security Assessment in Part 4 builds on these controls with a full review of attack paths and mitigations.

Benefit 5: Room to Grow and Change Quickly

On premises, adding capacity means buying hardware and waiting weeks. On AWS, the team can resize an EC2 instance, add a second instance behind the load balancer, or turn on a Multi-AZ database in minutes, and the $20,116 contingency already covers these upgrades. IDC found that IT teams deployed new compute resources 82% faster after moving to AWS (Carvalho & Marden, 2018). Because the team no longer maintains servers, it can spend more of its time building features that customers want.

3.5 How ACME Will Measure Success

Each benefit is tied to a target that the team tracks every sprint, using the performance measures from the Scrum Team Agreement (Section 1.5). Table 3 lists the targets ACME leadership can use to judge whether the investment is delivering.

Table 3. Success measures and targets

Benefit	Measure	Target
Lower costs	Annual AWS spend (AWS Budgets)	At or below $4,884 baseline; never above $25,000
Faster releases	Time from approved change to production	Under 1 hour, compared with a manual process today
Safer releases	Pipeline success rate and escaped defects	90% or higher success; 2 or fewer escaped defects per sprint
Availability	Monthly application uptime and recovery time	99.5% or higher uptime; rollback within 30 minutes
Security	Critical or high findings open at release	Zero

3.6 Recommendation

From a CEO’s point of view, this project offers a strong return for a low and controlled cost. It uses about one-fifth of the approved budget, delivers visible results every two weeks, and replaces a fragile manual process with one that is automated, secure, and measurable. The main risks, such as a single production server or traffic higher than expected, are already planned for and can be addressed within the remaining budget.

I recommend that ACME approve the project and begin Phase 1 (discovery and planning) in the next sprint. The first decisions needed from leadership are naming a Product Owner from ACME and confirming the database engine, so the team can start building the AWS foundation in the second sprint.

Part 4: Security Assessment

4.1 Project Overview

Item	Detail
Project name	ACME Cloud Corp Legacy Java Application Migration to AWS
Client	ACME Cloud Corp
Prepared by	Ernest Foubi, Project Manager, ECPI Consulting and Programming Institution
Environment	Cloud: Amazon Web Services (AWS), US East (N. Virginia) Region
Development method	Scrum (two-week sprints) with an automated CI/CD pipeline
Annual budget	$25,000 (baseline plan: $4,884 per year before the security services added in this assessment)

ACME Cloud Corp is moving its legacy Java web application from manually built, on-premises servers to AWS. This assessment identifies the most likely ways an attacker could harm the new environment, the controls that reduce each risk, the security policies the team will follow, and the plan the company will use to respond to an incident and return to normal operations. The development and security team members named in the Scrum Team Agreement developed it, and ACME leadership will approve it before the production cutover.

Expected Security Outcomes

- **Customer and company data stays confidential:** no sensitive data is exposed through stolen credentials, misconfiguration, or application flaws.
- **Only reviewed code reaches production:** every release pass peer review, automated tests, and security scans in AWS CodePipeline.
- **Attacks are detected quickly:** suspicious activity raises an alert within minutes instead of being found weeks later.
- **The business recovers fast:** the application can be restored to a well-good version and its data recovered to a recent point in time.
- **Security stays affordable:** the added security services cost about $60 per month, keeping the total plan near 22% of the $25,000 budget.

4.2 Technologies and Architecture in Scope

The assessment covers every component the attacker could reach or misuse. Under the AWS shared responsibility model, AWS secures the physical data centers, hardware, and core cloud services, while ACME is responsible for how it configures and protects its accounts, network, application, and data (Amazon Web Services, n.d.-i).

Layer	Technologies	Security role
Entry point	Application Load Balancer, AWS Certificate Manager, AWS WAF, AWS Shield Standard	Accepts HTTPS traffic only, filters malicious requests, absorbs network floods
Compute	Amazon EC2 (1 production, 1 staging), Linux, Java runtime	Runs the application in private subnets; no direct internet access to servers
Data	Amazon RDS (single-AZ), Amazon S3 (artifacts and backups), AWS Backup	Stores customer and business data, encrypted with AWS KMS
Identity	AWS IAM, IAM Identity Center with MFA, AWS Secrets Manager	Controls who and what can access each resource; stores passwords and keys
Delivery	GitHub, AWS CodePipeline, CodeBuild, CodeDeploy	Builds, tests, scans, and deploys every change
Monitoring	AWS CloudTrail, Amazon CloudWatch, Amazon GuardDuty, AWS Security Hub, AWS Config, Amazon Inspector	Records activity, detects threats and misconfigurations, raises alerts

Security Services Added by This Assessment

The baseline plan already included IAM, Secrets Manager, CloudWatch, and encryption. This assessment adds the services below. Costs are estimates based on AWS published pricing for an environment of this size and should be confirmed in the AWS Pricing Calculator.

Service	Purpose	Est. monthly cost
AWS WAF (web ACL with managed rule groups)	Blocks SQL injection, cross-site scripting, and known-bad requests; rate limits floods	$15
Amazon GuardDuty	Detects compromised credentials, unusual API calls, and malicious traffic	$15
AWS Security Hub and AWS Config	Checks the account against security best practices and flags misconfigurations	$15
Amazon Inspector	Scans EC2 instances for software vulnerabilities	$5
AWS CloudTrail log storage and cross-Region backup copies	Keeps tamper-resistant audit logs and off-site backups	$10
Total added		**$60 per month ($720 per year)**

With these services, the annual plan rises from $4,884 to about **$5,604**, roughly 22% of the budget, leaving more than $19,000 of headroom.

4.3 Threat Model

The team identified five threats by reviewing the architecture, the data the application handles, and common attacks on cloud-hosted Java applications, including the OWASP Top 10 (OWASP Foundation, 2021). Each threat is rated by its likelihood and potential impact. The table summarizes the model, and the subsections explain each attack avenue and its mitigations.

#	Threat	Avenue of attack	Likelihood / Impact
T1	Compromised credentials and privilege misuse	Phishing, reused passwords, AWS access keys leaked in code, overly broad IAM permissions	High / High
T2	Injection and web application attacks	SQL injection and cross-site scripting through forms and URLs; outdated Java libraries	Medium / High
T3	CI/CD pipeline and software supply chain compromise	Malicious or vulnerable dependency, stolen GitHub token, tampered build files or artifacts	Medium / High
T4	Cloud misconfiguration and data exposure	Public database or S3 bucket, open security group ports, unencrypted snapshots	Medium / High
T5	Denial of service	HTTP floods or bot traffic aimed at the load balancer and login page	Medium / Medium

T1: Compromised Credentials and Privilege Misuse

Avenue of attack: An attacker sends a phishing email to an administrator, reuses a password stolen in another breach, or finds an AWS access key accidentally committed to GitHub by a developer. With those credentials, the attacker could copy the RDS database, delete resources and backups, or launch crypto-mining servers that drive up ACME's bill. Because IAM controls everything in the account, this is the highest-rated threat.

Mitigations:

- Require multi-factor authentication for every user through IAM Identity Center; lock the root account with MFA and use it only for emergencies.

- Use IAM roles with temporary credentials instead of long-lived access keys and grant each person and service only the permissions it needs.
- Store database passwords and tokens in AWS Secrets Manager with automatic rotation; scan every commit for secrets in the pipeline.
- Turn on GuardDuty and CloudWatch alarms for root login, IAM policy changes, and unusual API activity; review user access every quarter.

T2: Injection and Web Application Attacks

Avenue of attack: Legacy Java applications often build database queries by joining strings together. An attacker can type SQL commands into a search box or login form to read or change the database (SQL injection), or plant scripts that run in other users' browsers (cross-site scripting). Older Java libraries can also contain known, easily exploited flaws, as the Log4Shell vulnerability in the Log4j library showed.

Mitigations:

- Rewrite database calls to use parameterized queries (prepared statements), validate all input, and encode output shown in web pages.
- Enable AWS WAF managed rule groups for SQL injection, cross-site scripting, and known bad inputs in front of the load balancer.
- Run static code analysis and dependency scanning in CodeBuild on every change; the build fails on critical or high findings.
- Give the application's database account only the permissions it needs, with no right to drop tables or change users.

T3: CI/CD Pipeline and Software Supply Chain Compromise

Avenue of attack: The pipeline can push code straight to production, which makes it a valuable target. An attacker could slip a malicious package into the application's Maven dependencies, steal the GitHub connection token, change the build specification to add a backdoor, or replace a tested artifact in S3 with a tampered one.

Mitigations:

- Protect the main branch: two peer approvals, no direct pushes, and review of every change to build and deployment files.
- Pin dependency versions, pull packages through a controlled repository such as AWS CodeArtifact, and scan dependencies for known vulnerabilities.
- Give CodePipeline, CodeBuild, and CodeDeploy separate least-privilege roles; encrypt the S3 artifact bucket, turn on versioning, and limit who can write to it.
- Keep the Product Owner's manual approval before production and log every pipeline change in CloudTrail.

T4: Cloud Misconfiguration and Data Exposure

Avenue of attack: Many cloud breaches come from settings, not hacking: a database marked publicly accessible, a security group that leaves SSH open to the whole internet, a public S3 bucket, or a database snapshot shared without encryption. Attackers run automated scans that find these mistakes within hours.

Mitigations:

- Keep EC2 and RDS in private subnets; allow database traffic only from the application's security group.
- Remove SSH access and use AWS Systems Manager Session Manager for administration, so no management port is open to the internet.
- Turn on S3 Block Public Access for the whole account and encrypt all storage, snapshots, and backups with AWS KMS by default.
- Use AWS Config rules and Security Hub best-practice checks to flag drift and build infrastructure from reviewed templates.

T5: Denial of Service

Avenue of attack: An attacker or botnet floods the load balancer with requests, or automated bots hammer the login page. Because the baseline design has a single production instance, a large flood could exhaust its CPU or database connections and take the application offline.

Mitigations:

- Rely on AWS Shield Standard for network-layer protection and add AWS WAF rate-based rules that block IP addresses sending too many requests.
- Set CloudWatch alarms on request counts, error rates, and CPU, and keep an Auto Scaling configuration ready to add a second instance behind the load balancer.
- Consider Amazon CloudFront in front of the load balancer if traffic grows, to absorb spikes closer to users.

4.4 Security Policies

The following policies turn the mitigations into daily habits. They are part of the team's Definition of Done, are reviewed at every Sprint Retrospective, and apply to ECPI and ACME staff working on the project.

Access Control

- Every person has an individual account with MFA; shared accounts are not allowed.
- Access is granted by role, on a least-privilege basis, and approved by the Project Manager and Security Engineer.
- Access is reviewed quarterly and removed within one business day when someone leaves the project.

Code Reviews

- Every pull request needs two approvals, including one reviewer who checks the security checklist: input validation, query handling, error messages, logging, and secrets.
- Automated static analysis, dependency, and secret scans must pass before review; reviewers do not approve code with open critical or high findings.
- Changes to IAM policies, network settings, or pipeline files also require approval from the DevOps or Security Engineer.

Architecture Reviews

- A security architecture review is held before the AWS foundation is built, before production cutover, and then every quarter, using the security pillar of the AWS Well-Architected Framework (Amazon Web Services, n.d.-h).
- Any major change, such as a new service, a public endpoint, or a change in data handling, triggers a review and an update to this threat model.

Testing Cadence

Security activity	Frequency	Owner
Unit, integration, and secret scanning	Every commit	Developers / pipeline
Static code analysis and dependency scanning	Every build	Pipeline / DevOps Engineer
Dynamic scan of staging (e.g., OWASP ZAP)	Every sprint	QA Engineer
Amazon Inspector vulnerability scans of EC2	Continuous; reviewed weekly	DevOps Engineer
Security Hub and AWS Config compliance review	Weekly	Security Engineer
Backup restore test	Quarterly	DevOps Engineer and DBA
Incident response tabletop exercise	Quarterly	Project Manager
Independent penetration test	Before go-live, then yearly	Security Engineer (third party)

Patching and Vulnerability Management

- Critical vulnerabilities are fixed within 7 days, high within 30 days, and medium within 90 days; actively exploited flaws are fixed immediately.
- Operating system patches are applied through AWS Systems Manager Patch Manager in staging first, then production.

Data Protection, Logging, and Backups

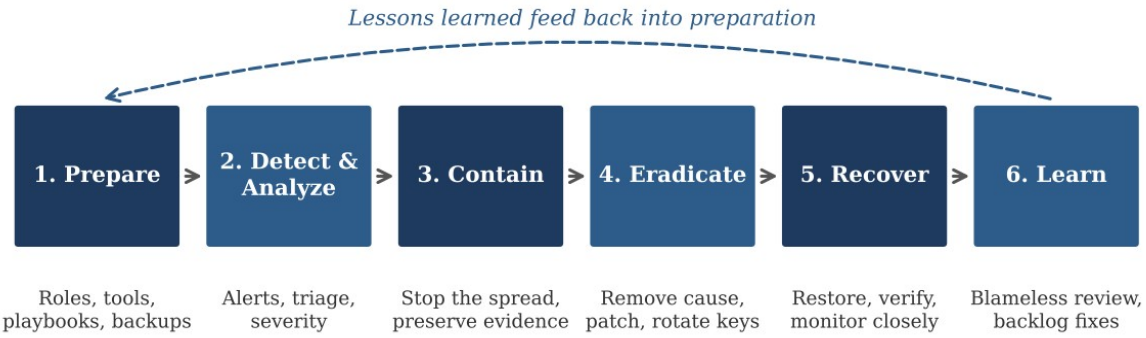
- All data is encrypted at rest with AWS KMS and in transit with TLS 1.2 or higher.
- CloudTrail records activity in all Regions; logs are kept for at least one year in an S3 bucket that cannot be altered or deleted.
- RDS keeps automated backups with point-in-time recovery for 14 days, and AWS Backup copies them to a second Region.

Security Awareness

- All team members complete security awareness and phishing training at onboarding and every year.

4.5 Incident Response Plan

This plan tells the team exactly what to do when a security incident happens, so that decisions are made quickly and calmly. It follows the incident response guidance in NIST SP 800-61 Revision 3, which aligns response activities with the NIST Cybersecurity Framework 2.0 (Nelson et al., 2025), and the AWS Security Incident Response Guide (Amazon Web Services, n.d.-g). The goal is to limit damage, correct the system, and return to normal operations as quickly and safely as possible.



Incident response lifecycle used by the ACME project team

Incident Response Team and Roles

Role	Assigned to	Responsibilities
Incident Commander	Project Manager / Scrum Master	Leads the response, sets severity, assigns tasks, approves major actions, and keeps the timeline
Technical Lead	DevOps / Cloud Engineer	Investigates AWS resources, contains and fixes systems, and runs recovery
Security Lead	Security Engineer	Analyzes evidence, determines root cause and scope, and advises on containment
Application and Data Support	Java Lead, DBA, QA Engineer	Fix code, restore and validate the database, and test before release
Communications Lead	Product Owner (ACME)	Updates ACME leadership, customers, and partners using approved messages
Legal and Compliance	ACME legal counsel	Decides on breach notifications and contact with law enforcement

Contact details for every role, plus AWS Support, are kept in the incident runbook and in a printed copy, so the team can reach each other even if company systems are down.

Severity Levels

Severity	Examples	Response start	Leadership updates
SEV 1 – Critical	Confirmed data breach, production down, ransomware, attacker inside the AWS account	15 minutes, 24/7	Every hour
SEV 2 – High	Suspected credential compromise, active attack being blocked, partial outage	1 hour	Every 4 hours
SEV 3 – Medium	Malware on one instance, misconfiguration found with no sign of misuse	4 business hours	Daily
SEV 4 – Low	Failed attack attempts, minor policy violation	Next business day	In weekly report

Phase 1: Preparation

Preparation happens before anything goes wrong. The team keeps this plan and the playbooks below in the runbook, confirms that GuardDuty, CloudTrail, CloudWatch alarms, and Security Hub send alerts to an on-call email and Microsoft Teams channel, and maintains a pre-approved “break-glass” administrator account protected by MFA for emergencies. Backups are tested every quarter, and the team practices the plan in a tabletop exercise every quarter.

Phase 2: Detection and Analysis

Incidents are detected through GuardDuty findings, CloudWatch alarms, WAF and load balancer logs, failed pipeline security checks, or reports from users and staff. The first responder opens an incident ticket in Jira and notifies the Incident Commander, who sets the severity. The Security Lead then confirms whether the event is a real incident, identifies affected accounts, instances, and data, and records every finding and action with a timestamp. The team preserves evidence such as CloudTrail logs, application logs, and EBS snapshots of affected servers before making any changes.

Phase 3: Containment

Containment stops the damage from spreading while keeping evidence intact. Short-term actions are taken right away, for example isolating an EC2 instance by moving it to a security group that allows no traffic, disabling a compromised user or access key, or blocking attacking IP addresses in AWS WAF. Long-term

containment keeps the business running safely, for example by routing traffic to a clean instance launched from a known-good image while the affected one is investigated.

Phase 4: Eradication

Once the incident is contained, the team removes its cause. This includes deleting malicious code or unauthorized resources, rotating every password, key, and token that may have been exposed, patching the vulnerability that was exploited, and correcting misconfigurations. Compromised servers are not cleaned and reused; they are replaced with new instances built through the pipeline. The Security Lead confirms that no backdoors or unauthorized IAM users or roles remain.

Phase 5: Recovery and Return to Normal Operations

Recovery restores full service in a controlled way. The team redeploys the last known-good application version through CodePipeline and CodeDeploy, and if data was changed or deleted, restores RDS to a point in time just before the incident. QA runs smoke and functional tests in staging before traffic returns to production. After recovery, the team watches the environment closely for at least 72 hours. The Incident Commander declares the incident closed only when all of the following are true:

- The application passes health checks and functional tests, and response times are back to normal.
- Data has been validated against backups, and no unauthorized changes remain.
- All exposed credentials have been rotated, and the exploited weakness is fixed.
- Monitoring shows no new suspicious activity, and the Product Owner confirms normal business operations.

Phase 6: Post-Incident Review

Within five business days, the team holds a blameless review to document what happened, how it was detected, how long each phase took, and what would prevent it next time. The team adds improvements to the product backlog with owners and due dates, and updates this plan, the threat model, and the playbooks. Key measures, including time to detect, time to contain, and time to recover, are reported to ACME leadership.

Threat-Specific Playbooks

The playbooks below give the first actions for each threat in the threat model, so responders do not have to decide under pressure.

Threat	Contain	Eradicate	Recover
T1 Compromised credentials	Disable the user and access keys; revoke active sessions; restrict the account with a deny policy if needed	Rotate all secrets; delete unauthorized users, roles, and resources; review CloudTrail for every action taken	Restore deleted resources from templates and backups; re-enable access with new MFA; confirm billing is normal
T2 Injection attack	Block attacking IPs and patterns in WAF; take the vulnerable feature offline if needed	Fix the code with parameterized queries; patch outdated libraries; rescan	Deploy the fix through the pipeline; check the database for changed or stolen records
T3 Pipeline compromise	Pause the pipeline; revoke the GitHub connection and pipeline credentials	Remove malicious commits or packages; rebuild from a verified commit; rotate tokens	Redeploy a verified artifact; compare production with the last known-good version
T4 Misconfiguration / data exposure	Remove public access; close open ports; stop sharing exposed snapshots	Correct the setting through a reviewed template; add an AWS Config rule to catch it	Review access logs to see what was exposed; notify affected parties if data was accessed
T5 Denial of service	Tighten WAF rate limits; block attacking sources; engage AWS Support	Adjust WAF rules for the attack pattern	Scale out behind the load balancer if needed; return to normal limits once traffic settles

Communication and Notification

The Communications Lead sends updates at the intervals set by the severity level, and the team uses a dedicated Teams channel, so information is not lost in other conversations. Outside parties hear only from approved spokespeople. If personal data may have been exposed, ACME legal counsel decides whether to notify customers, partners, or regulators under applicable state breach-notification laws and contracts, and the team provides the facts needed to make that decision quickly.

References

- Amazon Web Services. (n.d.-a). *AWS Certificate Manager user guide*. https://docs.aws.amazon.com/acm/latest/userguide/acm-overview.html
- Amazon Web Services. (n.d.-b). *AWS CodeBuild user guide*. https://docs.aws.amazon.com/codebuild/latest/userguide/welcome.html
- Amazon Web Services. (n.d.-c). *AWS CodeDeploy user guide*. https://docs.aws.amazon.com/codedeploy/latest/userguide/welcome.html
- Amazon Web Services. (n.d.-d). *AWS CodePipeline user guide*. https://docs.aws.amazon.com/codepipeline/latest/userguide/welcome.html
- Amazon Web Services. (n.d.-e). *AWS pricing*. https://aws.amazon.com/pricing/
- Amazon Web Services. (n.d.-f). *AWS Pricing Calculator*. https://calculator.aws/
- Amazon Web Services. (n.d.-g). *AWS security incident response guide*. https://docs.aws.amazon.com/whitepapers/latest/aws-security-incident-response-guide/welcome.html
- Amazon Web Services. (n.d.-h). *Security pillar: AWS Well-Architected Framework*. https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/welcome.html
- Amazon Web Services. (n.d.-i). *Shared responsibility model*. https://aws.amazon.com/compliance/shared-responsibility-model/
- Carvalho, L., & Marden, M. (2018). *Fostering business and organizational transformation to generate business value with Amazon Web Services* (Doc. #US43535718). IDC. https://pages.awscloud.com/IDC-Fostering-Business-and-Organizational-Transformation-to-Generate-Business-Value-with-AWS.html
- Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The science of lean software and DevOps: Building and scaling high-performing technology organizations*. IT Revolution Press.
- Forsgren, N., Smith, D., Humble, J., & Frazelle, J. (2019). *2019 Accelerate state of DevOps report*. DORA and Google Cloud. https://services.google.com/fh/files/misc/state-of-devops-2019.pdf
- Nelson, A., Rekhi, S., Souppaya, M., & Scarfone, K. (2025). *Incident response recommendations and considerations for cybersecurity risk management: A CSF 2.0 community profile* (NIST Special

Publication 800-61r3). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-61r3

- OWASP Foundation. (2021). *OWASP Top 10: 2021*. https://owasp.org/Top10/
- Schwaber, K., & Sutherland, J. (2020). *The Scrum guide: The definitive guide to Scrum: The rules of the game*. https://scrumguides.org/scrum-guide.html